

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
"НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ НАУК**

Казанцев Андрей Григорьевич

Выпускная квалификационная работа – МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ

**АНАЛИТИЧЕСКИЙ ДАШБОРД ДЛЯ ПРИНЯТИЯ УПРАВЛЕНЧЕСКИХ РЕШЕНИЙ И
РАСЧЕТА МАРЖИНАЛЬНОСТИ НА МАРКЕТПЛЕЙСАХ С ИСПОЛЬЗОВАНИЕМ API-
ИНТЕГРАЦИЙ И АЛГОРИТМОВ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА**

**ANALYTICAL DASHBOARD FOR MANAGERIAL DECISION-MAKING AND PROFIT-
MARGIN CALCULATION ON E-COMMERCE MARKETPLACES USING API
INTEGRATIONS AND ARTIFICIAL INTELLIGENCE ALGORITHMS**

по направлению подготовки 01.04.02 «Прикладная математика и информатика»
образовательная программа «Искусственный интеллект в маркетинге и управлении
продуктом»

Рецензент

Иволга Т.А.

Научный руководитель

Старший преподаватель
Паточенко Е. А.

Москва, 2026

СОДЕРЖАНИЕ	
АННОТАЦИЯ.....	3
ВВЕДЕНИЕ.....	4
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ РЕШЕНИЙ.....	8
1.1. Особенности торговли на маркетплейсах и структура издержек продавца	8
1.2. Подходы к расчёту юнит-экономики в электронной коммерции	10
1.3. Обзор существующих аналитических решений	11
1.4. Подходы к планированию поставок при работе по модели FBO	13
1.5. Применение методов искусственного интеллекта в электронной коммерции	15
Выводы по главе 1	16
ГЛАВА 2. ПРОЕКТИРОВАНИЕ И АРХИТЕКТУРА СИСТЕМЫ	18
2.1. Требования к системе.....	18
2.2. Архитектура решения	19
2.3. Модель данных	20
2.4. Проектирование AI-компонента	21
2.5. Выбор технологического стека	21
Выводы по главе 2	22
ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ АНАЛИТИЧЕСКОГО ПРИЛОЖЕНИЯ	
ALSCHEMY	23
3.1. Общая структура проекта	23
3.2. Реализация получения данных через API маркетплейсов	26
3.2.1. Интеграция с Ozon	26
3.2.2. Интеграция с Wildberries.....	26
3.2.3. Интеграция с Яндекс Маркетом	27
3.2.4. Хранение секретов	27
3.3. Реализация логики расчёта юнит-экономики	27
3.3.1. Общая расчётная модель	28
3.3.2. Особенности применения модели для разных маркетплейсов	29
3.3.3. Сопоставимость итогов между площадками.....	30
3.4. Реализация AI-компонента	30
3.5. Пользовательский интерфейс	31
3.6. Сценарий работы пользователя.....	35
3.7. Проверка корректности расчётов и апробация приложения.....	36
3.8. Ограничения текущей реализации и направления развития	37
Выводы по главе 3	38
ЗАКЛЮЧЕНИЕ	40
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	42

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке аналитического дашборда для продавцов маркетплейсов, предназначенного для расчёта маржинальности, анализа структуры затрат, поддержки управленческих решений и планирования поставок. Актуальность работы обусловлена сложностью расчёта фактического финансового результата на маркетплейсах, где итоговая прибыльность товара зависит от комиссий, логистики, возвратов, рекламных расходов, скидочных механизмов и различий в отчётности площадок. В работе проанализированы особенности торговли по модели FBO, подходы к расчёту юнит-экономики и существующие аналитические решения для продавцов. На основе проведённого анализа спроектирована и реализована модульная система Alchemy, включающая интеграции с API Ozon, Wildberries и Яндекс Маркета, расчётные модули юнит-экономики, кэширование данных, пользовательский интерфейс на базе Streamlit и AI-компонент для интерпретации рассчитанных показателей. Программная реализация доведена до серверной версии, развернутой на собственном сервере.

Ключевые слова: маркетплейсы, юнит-экономика, маржинальность, API-интеграция, Ozon, Wildberries, Яндекс Маркет, Streamlit, аналитический дашборд, искусственный интеллект.

ВВЕДЕНИЕ

В последние годы электронная коммерция занимает всё более значимое место в структуре розничной торговли [5, 6]. Для многих компаний малого и среднего бизнеса маркетплейсы стали важным каналом продаж, обеспечивающим доступ к широкой аудитории покупателей и готовой логистической инфраструктуре. Вместе с тем работа на таких площадках, как Ozon, Wildberries и Яндекс Маркет, имеет свою специфику и связана с постоянным увеличением сложности расчетов и планирования.

В научной литературе формирование предпринимательской стратегии на маркетплейсах рассматривается как циклический процесс, основанный на регулярном анализе цифровых метрик и корректировке управленческих решений в условиях алгоритмически управляемой среды [18]. Реализация такого подхода требует от продавца постоянного доступа к актуальным показателям маржинальности, структуры затрат и эффективности по отдельным товарным позициям, что обосновывает потребность в специализированных аналитических инструментах.

Одной из наиболее существенных проблем при работе на маркетплейсах является сложность оперативной оценки фактической прибыли. Продавец видит цену реализации, размер скидок и объём продаж, однако сумма фактических начислений по товару формируется под влиянием множества факторов: комиссии маркетплейса, расходов на логистику, магистральную доставку, эквайринг, продвижение, а также затрат, связанных с возвратами и невыкупам. В результате выручка по товару не всегда отражает его реальную маржинальность, а принятие управленческих решений нередко осуществляется на основе неполных или прогнозных данных. Это может приводить к сохранению в ассортименте убыточных позиций, неэффективному распределению рекламного бюджета и ошибкам в ценообразовании.

Дополнительную сложность представляет планирование поставок при использовании складской инфраструктуры маркетплейсов. Крупные площадки, такие как Ozon и Wildberries, располагают распределённой сетью складов, охватывающей различные регионы страны. Размещение товара на нескольких складах позволяет сократить сроки доставки до покупателя и снизить логистические расходы за счёт локализации запасов ближе к точкам спроса. Однако эффективное управление такой сетью требует от продавца понимания географической структуры продаж и способности прогнозировать спрос в разрезе регионов.

На практике продавцы нередко ориентируются на статистику оборачиваемости по складам отгрузки. Такой подход может приводить к искажённому представлению о структуре спроса. Если товар отгружается с московского склада покупателю в Екатеринбурге, продажа учитывается за московским складом, хотя фактический спрос сформировался в другом регионе. В результате продавец может недооценивать потребность в размещении товара на региональных складах, что ведёт к росту затрат на магистраль, увеличению сроков доставки и снижению

конкурентоспособности карточки товара в региональной выдаче, а также увеличению комиссии маркетплейса за нелокальную продажу.

Маркетплейсы, в свою очередь, заинтересованы в повышении локализации запасов и применяют различные механизмы стимулирования: пониженные тарифы на логистику при отгрузке с ближайшего к покупателю склада, повышенную видимость локализованных товаров в поисковой выдаче, а также штрафные коэффициенты за избыточный запас товаров на складах. В этих условиях корректное распределение запасов по складской сети становится не только задачей снижения издержек, но и фактором, влияющим на объём продаж. Следовательно, для обоснованного планирования поставок необходим инструмент, позволяющий анализировать не только склад отгрузки, но и регион конечного спроса, выявлять устойчивые межрегиональные потоки и формировать рекомендации по размещению товара.

На рынке уже представлены различные инструменты аналитики для продавцов маркетплейсов. К первой группе относятся сервисы внешней аналитики, такие как MPStats, Moneyplace, MarketGuru и другие, позволяющие анализировать позиции в выдаче, категории и конкурентную среду. Однако подобные решения, как правило, не фокусируются на внутренних данных конкретного продавца и потому ограничены применимостью для точного расчёта собственной прибыльности.

Ко второй группе относятся интеграции с учётными системами и ERP-решения, в том числе на базе 1С. Они обеспечивают более глубокую автоматизацию, но часто оказываются избыточными по сложности, стоимости внедрения и сопровождения для малого и среднего бизнеса.

Третью группу составляют специализированные сервисы расчёта прибыли. Несмотря на функциональную близость к поставленной задаче, такие решения нередко ориентируются на расчёты по заказам, а не по фактическим начислениям площадки, что может снижать точность итоговых показателей. Кроме того, многие из них обладают избыточной функциональностью, усложняющей практическое использование.

Развитие методов искусственного интеллекта создаёт дополнительные возможности для совершенствования аналитических инструментов. Современные языковые модели могут использоваться не только для обработки текстовой информации, но и для интерпретации структурированных данных, выявления проблемных зон и формирования аналитических рекомендаций в понятной для пользователя форме. Это особенно важно для продавцов, не обладающих глубокой подготовкой в области анализа данных [10].

Таким образом, актуальность данной работы обусловлена необходимостью создания инструмента, который обеспечивал бы более точный расчёт маржинальности товаров на

маркетплейсах, поддержку принятия решений на основе фактических данных и более обоснованное планирование поставок с учётом географии спроса.

Целью выпускной квалификационной работы является разработка аналитического приложения для продавцов маркетплейсов, предназначенного для расчёта маржинальности по каждому SKU, анализа структуры затрат, поддержки управленческих решений по ассортименту и ценообразованию, а также планирования поставок с учётом регионального распределения спроса.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) проанализировать предметную область, включая структуру издержек продавца на маркетплейсах Ozon и Wildberries, а также существующие подходы и инструменты аналитики;
- 2) спроектировать архитектуру модульной системы, обеспечивающей интеграцию с API нескольких маркетплейсов;
- 3) реализовать модули автоматического сбора данных и расчёта юнит-экономики по каждому SKU на основе фактических данных из отчётов о начислениях;
- 4) разработать модуль планирования поставок с учётом географии спроса и распределения продаж по регионам доставки;
- 5) интегрировать AI-компонент на базе языковой модели для генерации аналитических рекомендаций по результатам расчётов;
- 6) провести верификацию расчётов путём сопоставления результатов системы с официальными отчётами маркетплейсов и выполнить апробацию решения на реальных данных.

Объектом исследования является процесс управления ассортиментом, поставками и финансовыми показателями продавца на маркетплейсах.

Предметом исследования являются методы и инструменты автоматизированного расчёта маржинальности, анализа структуры затрат и аналитической поддержки принятия решений в электронной коммерции.

Научная новизна работы заключается в следующем. Во-первых, расчёт маржинальности выполняется на основе фактических данных о начислениях маркетплейса, а не на основе данных о заказах. Во-вторых, при планировании поставок учитывается регион доставки как характеристика фактического спроса, а не только склад отгрузки. В-третьих, в систему интегрирован AI-компонент, обеспечивающий интерпретацию результатов расчётов и формирование рекомендаций для пользователя.

Практическая значимость работы состоит в том, что разработанная система апробирована на реальных данных действующей компании и доведена до серверной версии, развернутой на собственном сервере автора. Приложение используется сотрудниками в рабочих аналитических сценариях и может применяться продавцами маркетплейсов для повышения прозрачности

финансовых показателей, выявления убыточных позиций, оптимизации поставок и ускорения принятия управленческих решений.

Структура работы включает введение, три главы, заключение и список использованных источников. В первой главе рассматриваются особенности предметной области и существующие аналитические решения. Во второй главе описываются проектирование и архитектура системы. Третья глава посвящена программной реализации приложения.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ РЕШЕНИЙ

1.1. Особенности торговли на маркетплейсах и структура издержек продавца

Деятельность продавца на маркетплейсах имеет ряд особенностей, отличающих её от традиционных форм электронной торговли. Эти особенности связаны не только с использованием цифровой витрины для размещения товаров, но и с участием маркетплейса в логистике, расчётах, ценообразовании и продвижении продукции. В результате продавец функционирует в условиях сложной организационно-экономической среды, где итоговая эффективность определяется совокупностью финансовых, логистических и операционных факторов.

Одной из ключевых характеристик работы на маркетплейсах является наличие нескольких моделей сотрудничества, определяющих распределение обязанностей между площадкой и продавцом. Наиболее распространёнными являются модели FBO, FBS и DBS.

Модель FBO (Fulfillment by Operator) предполагает поставку товара на склад маркетплейса, после чего функции хранения, комплектации, доставки покупателю и обработки возвратов выполняются силами площадки. В рамках модели FBS (Fulfillment by Seller) товар находится на складе продавца, а после поступления заказа передаётся в логистическую систему маркетплейса для дальнейшей доставки. Модель DBS (Delivery by Seller) предусматривает, что продавец самостоятельно осуществляет как хранение, так и доставку товара, тогда как маркетплейс обеспечивает размещение товара на витрине и привлечение заказов.

Выбор модели сотрудничества оказывает непосредственное влияние на структуру бизнес-процессов и состав издержек продавца. Для целей настоящего исследования наибольший интерес представляет модель FBO, поскольку именно при её использовании особенно значимыми становятся вопросы распределения товарных запасов по складской сети маркетплейса, анализа перемещений между регионами и планирования поставок с учётом пространственного распределения спроса.

Существенной особенностью торговли на маркетплейсах является многоуровневая структура затрат. В состав расходов продавца могут входить комиссия площадки, затраты на логистику, магистральные перемещения между складами, обратную логистику при возвратах и невыкупках, хранение товаров, эквайринг, продвижение, а также штрафы и иные удержания. Конкретный объём и структура расходов зависят от категории товара, выбранной модели сотрудничества, участия в акциях, региона доставки, весогабаритных характеристик продукции и других параметров.

Дополнительные трудности связаны с тем, что разные маркетплейсы используют различные подходы к расчёту комиссий и представлению отчётных данных. Например, Ozon и

Wildberries отличаются по структуре тарифов, принципам начисления отдельных видов расходов и формату финансовой отчётности. Одни и те же по экономическому содержанию затраты могут быть представлены в отчётах площадок по-разному: как отдельные статьи либо как элементы укрупнённых показателей. Это затрудняет сопоставление финансовых результатов при одновременной работе на нескольких маркетплейсах и усложняет получение единой аналитической картины по ассортименту.

Особое значение имеет специфика формирования фактического финансового результата по товару. На маркетплейсах продавец сталкивается с ситуацией, когда цена, отображаемая покупателю, не совпадает напрямую с суммой, которая в конечном итоге подлежит перечислению продавцу. Это обусловлено использованием различных механизмов скидок, акций, программ софинансирования и иных ценовых инструментов. Вследствие этого фактические начисления по товару формируются на основе внутренних правил площадки и включают целый комплекс удержаний и корректировок. В таких условиях данные о выручке и количестве продаж сами по себе недостаточны для объективной оценки прибыльности конкретной товарной позиции.

Не менее важной особенностью является различие между складом отгрузки и регионом фактического спроса. На практике анализ продаж нередко осуществляется в разрезе складов, с которых производилась отгрузка заказов. Однако такой подход не всегда позволяет корректно определить географию потребления. Если товар был отправлен со склада, расположенного в одном регионе, покупателю, находящемуся в другом регионе, то продажа будет отражена по месту отгрузки, тогда как фактический спрос сформировался в иной территориальной зоне.

Подобная ситуация может приводить к искажению представлений о распределении спроса и, как следствие, к ошибкам в планировании поставок. Недостаточный учёт региональной структуры заказов способен вызывать нерациональное размещение товарных запасов, увеличение объёма магистральных перемещений, рост логистических издержек и снижение скорости доставки. Кроме того, это может отрицательно влиять на конкурентные позиции товара в региональной выдаче, поскольку маркетплейсы, как правило, заинтересованы в локализации запасов ближе к покупателю.

Таким образом, для корректной оценки эффективности продаж необходимо учитывать не только объёмы реализации, но и фактическую структуру начислений, особенности логистических процессов, различия в отчётности площадок и географию спроса. Это обуславливает необходимость разработки специализированного аналитического инструмента, способного обеспечить более точный расчёт маржинальности и поддержку решений в области управления ассортиментом и поставками.

1.2. Подходы к расчёту юнит-экономики в электронной коммерции

Юнит-экономика представляет собой подход к анализу бизнеса, при котором в качестве базовой единицы рассматривается отдельная продажа, товарная позиция или транзакция. В условиях торговли на маркетплейсах применение данного подхода позволяет оценивать экономическую эффективность каждого SKU, выявлять убыточные или низкомаржинальные позиции, а также принимать обоснованные решения в области ценообразования, продвижения и логистики [9, 10].

Для расчёта юнит-экономики необходимо определить систему базовых показателей. В контексте маркетплейсов под выручкой может пониматься как брутто-выручка, то есть стоимость товара до удержаний, так и нетто-выручка, представляющая собой сумму, остающуюся после вычета комиссий, логистических расходов, скидок, затрат на продвижение и иных списаний. С точки зрения управленческого анализа более информативным является именно нетто-подход, поскольку он позволяет оценить фактический денежный результат продажи. В данной работе в качестве основы расчёта используется нетто-выручка.

Маржа определяется как разница между нетто-выручкой и себестоимостью товара. Маржинальность, в свою очередь, представляет собой относительный показатель, характеризующий долю маржи в выручке. Использование этих показателей позволяет сопоставлять товарные позиции различных ценовых категорий и выявлять товары, которые при высоком объёме продаж демонстрируют недостаточную экономическую эффективность.

Отдельное значение в рамках анализа юнит-экономики имеет доля рекламных расходов (ДРР), отражающая отношение затрат на продвижение к выручке. Анализ данного показателя позволяет оценивать эффективность рекламной активности на уровне отдельного SKU и принимать решения о целесообразности дальнейшего продвижения товара, корректировке рекламной стратегии или перераспределении бюджета.

В практике расчёта юнит-экономики на маркетплейсах можно выделить два основных подхода.

Первый подход основан на данных о заказах и прогнозных тарифах площадки. В данном случае расчёт выполняется исходя из цены продажи, из которой вычитаются комиссии и расходы, определяемые по действующим условиям маркетплейса. Преимуществом такого подхода является высокая оперативность, поскольку сведения о заказах доступны практически в реальном времени. Вместе с тем его точность ограничена тем, что прогнозные значения могут заметно отличаться от фактических начислений. При данном способе расчёта невозможно в полной мере учесть возвраты, невыкупы и изменение логистических условий.

Второй подход основан на фактических начислениях маркетплейса. В этом случае расчёт строится на основе полученных от покупателей денежных средств, за минусом удержанных сумм

комиссий, логистических расходов, затрат на хранение, эквайринг, продвижение и иных удержаний. Данный подход обеспечивает более высокую точность, поскольку опирается на фактически сформированный финансовый результат. Его основным ограничением является наличие временного лага между моментом совершения продажи и доступностью полной отчётности.

В настоящей работе применяется второй подход - расчёт на основе фактически начисленных сумм. Источником данных для Ozon служит транзакционная отчётность, получаемая через метод `/v3/finance/transaction/list` Seller API. Для Wildberries основным источником служит детализированный отчёт о реализации, получаемый через метод `/api/v5/supplier/reportDetailByPeriod`. Данный отчёт позволяет формировать как оперативные срезы продаж за отдельные периоды, так и сводные значения за более длительный интервал, что используется для верификации итоговых показателей. Информация о рекламных расходах извлекается через специализированные рекламные API соответствующих маркетплейсов, что позволяет учитывать затраты на продвижение при расчёте маржинальности.

Выбор фактического подхода обусловлен тем, что целью работы является не прогнозная оценка прибыльности, а получение максимально точного представления о маржинальности каждого SKU на основе реальных данных маркетплейсов с начала текущего месяца и за вчерашний день. Такой подход в наибольшей степени соответствует задачам управленческого анализа, поскольку позволяет принимать решения, опираясь на фактически сформированные показатели, а не на расчётные предположения.

1.3. Обзор существующих аналитических решений

В настоящее время продавцы на маркетплейсах используют различные инструменты для анализа продаж, расчёта прибыльности и поддержки управленческих решений. Эти инструменты различаются по уровню автоматизации, точности расчётов, стоимости внедрения и удобству практического применения.

Одним из наиболее распространённых способов, особенно среди небольших продавцов, остаётся ручной расчёт в электронных таблицах. Такой подход предполагает выгрузку нескольких отчётов из личных кабинетов маркетплейсов и их последующую обработку в Excel или Google Sheets. При использовании отчётов по начислениям данный способ может обеспечивать достаточно высокую точность расчётов. Вместе с тем он требует значительных временных затрат, уверенного владения табличными редакторами и аккуратной ручной консолидации данных. При одновременной работе на нескольких площадках трудоёмкость

такого подхода существенно возрастает, а вероятность ошибок увеличивается. В связи с этим ручной расчёт ограниченно подходит для регулярного оперативного анализа и масштабирования.

Другую группу решений составляют встроенные калькуляторы маркетплейсов. Они позволяют оценить предполагаемую прибыльность товара с учётом комиссий и логистических расходов в рамках конкретной площадки. К их основным преимуществам относятся доступность и отсутствие дополнительных затрат на внедрение. Однако такие калькуляторы, как правило, используют прогнозные значения и ориентированы только на одну платформу. В результате продавец не получает единого инструмента для сопоставления показателей одного и того же товара на разных маркетплейсах и комплексного анализа эффективности ассортимента.

Существенную часть рынка занимают специализированные сервисы аналитики маркетплейсов. К ним относятся как сервисы внешней аналитики и конкурентного мониторинга, например MPStats, Moneyplace, MarketGuru и Shopstat, так и сервисы агрегации отчётности и расчёта прибыли, включая Маяк, Анабар, SellerBoard и JVO. Несмотря на различия в позиционировании и функциональных возможностях, данные решения можно разделить на несколько основных групп.

Сервисы внешней аналитики ориентированы преимущественно на исследование конкурентной среды, анализ динамики категорий, мониторинг позиций товаров в поисковой выдаче и поиск перспективных ниш. Некоторые из них поддерживают подключение кабинета продавца и расчёт дополнительных показателей, однако их основное назначение связано не с точным расчётом юнит-экономики по внутренним данным продавца, а с анализом рыночной ситуации и конкурентной активности.

Сервисы агрегации отчётности и расчёта прибыли в большей степени соответствуют задачам внутренней аналитики, поскольку используют данные из личных кабинетов продавца. Вместе с тем обзор существующих решений показывает, что подобные сервисы нередко характеризуются высокой стоимостью, значительной функциональной сложностью либо ограниченной адаптацией к специфике российских маркетплейсов. Кроме того, в ряде случаев расчёты выполняются не на основе фактических начислений, а на данных о заказах или расчётных величинах, что может снижать точность итоговых показателей.

В академической литературе рассматриваются более сложные подходы к управлению эффективностью продаж на маркетплейсах. В частности, изучаются модели динамического ценообразования на основе алгоритмов машинного обучения [12], а также методы оптимизации цен на маркетплейсе Ozon с использованием градиентного бустинга и линейного программирования [13]. Подобные подходы пока не получили широкого распространения в коммерческих сервисах для продавцов, что подчёркивает целесообразность развития

прикладных аналитических инструментов, ориентированных на задачи юнит-экономики и поддержки управленческих решений.

Отдельную категорию составляют интеграции с учётными системами, включая решения на базе 1С, «МойСклад» и других ERP-платформ. Их преимуществом является глубокая интеграция с внутренними бизнес-процессами предприятия, в том числе с управлением остатками, закупками, складским учётом и финансовой отчётностью. Однако внедрение и сопровождение таких систем требует значительных временных, организационных и финансовых ресурсов, связанных с настройкой обмена данными, поддержкой интеграций и обучением персонала. Для малых и части средних продавцов подобные решения нередко оказываются избыточными по стоимости и сложности эксплуатации.

Проведённый обзор показывает, что на рынке представлены инструменты, позволяющие решать отдельные аналитические задачи продавца, включая внешнюю аналитику, консолидацию отчётности, расчёт прибыли и интеграцию с учётными системами. Однако решения, сочетающие расчёт юнит-экономики по нескольким маркетплейсам на основе фактических начислений, анализ структуры затрат на уровне SKU, поддержку планирования поставок с учётом географии спроса и удобную интерпретацию результатов, представлены ограниченно. Это обосновывает целесообразность разработки специализированного аналитического приложения, ориентированного на решение указанных задач.

1.4. Подходы к планированию поставок при работе по модели FBO

При работе по модели FBO продавец размещает товар на складах маркетплейса, а дальнейшие операции по хранению, комплектации и доставке заказа осуществляет сама площадка. В этих условиях особое значение приобретает задача планирования поставок, то есть определения того, на какие склады, в каком объёме и в какие сроки необходимо направлять товар. От качества такого планирования зависит не только наличие товара в продаже, но и общая эффективность его реализации.

Корректное распределение запасов по складам особенно важно по нескольким причинам. Во-первых, наличие товара в регионе, близком к покупателю, положительно влияет на его ранжирование в локальной выдаче маркетплейса. Во-вторых, размещение товара ближе к конечному спросу позволяет снижать отдельные логистические издержки, включая расходы, связанные с межрегиональными перемещениями. В-третьих, сокращение расстояния между складом и покупателем способствует ускорению доставки, что также положительно влияет на вероятность покупки и общий объём продаж. Таким образом, при работе по модели FBO размещение запасов по складам становится не только логистической, но и коммерческой задачей.

На практике продавцы используют несколько основных подходов к планированию поставок. Наиболее распространённым является подход, основанный на анализе

оборачиваемости и продаж по складам. В этом случае решение о пополнении запасов принимается исходя из того, на каком складе товар быстрее продаётся или быстрее заканчивается. Такой подход удобен, поскольку соответствующие показатели доступны в личном кабинете маркетплейса и не требуют сложной дополнительной обработки.

Однако у данного подхода имеется существенное ограничение. Продажи в разрезе складов отражают прежде всего склад отгрузки, а не географию конечного спроса. Если заказ был отгружен, например, с одного крупного склада в другой регион, то такая продажа будет учтена именно за складом отгрузки. При этом в стандартных отчётах, доступных продавцу для ручной выгрузки, как правило, отсутствуют данные, позволяющие в явном виде определить конечный кластер или регион доставки заказа. В результате оборачиваемость по складу не всегда позволяет сделать корректный вывод о том, где именно товар реально востребован. Это может приводить к ошибкам в распределении запасов и к повторному усилению тех складов, которые используются как основные точки отгрузки, но не обязательно соответствуют зонам фактического спроса.

Вторым распространённым вариантом является использование готовых планов поставок, формируемых самим маркетплейсом. Такие рекомендации удобны с практической точки зрения, поскольку позволяют продавцу быстро получить ориентир по объёму и направлению поставки без самостоятельного анализа большого массива данных. Однако логика формирования подобных рекомендаций для продавца, как правило, остаётся непрозрачной. Пользователь не может в полной мере оценить, на каких именно данных и критериях основан предложенный план, какие факторы были учтены при его построении и насколько рекомендации согласуются с особенностями конкретного ассортимента. Это снижает управляемость процесса и ограничивает возможность критической проверки предлагаемых решений.

Третьим подходом является использование специализированных инструментов и функций аналитики, предоставляемых маркетплейсами или внешними сервисами. Такие решения могут включать анализ остатков, скорости продаж, оборачиваемости и рекомендации по пополнению отдельных складов. Несмотря на более высокий уровень автоматизации, в основе подобных инструментов нередко также лежат показатели по складам отгрузки и текущему движению запасов. Следовательно, при отсутствии достоверных данных о конечной географии спроса даже такие решения могут воспроизводить те же ограничения, что и ручной анализ оборачиваемости.

В академической литературе для задач планирования поставок и управления запасами рассматривается применение методов машинного обучения, включая нейросетевые модели прогнозирования временных рядов и алгоритмы градиентного бустинга [17]. Подобные подходы позволяют учитывать сезонные колебания, тренды и нелинейные закономерности спроса. Однако

их эффективное применение требует доступа к достоверным данным о конечной географии спроса, что возвращает к описанной выше проблеме источников информации.

Таким образом, существующие подходы к планированию поставок при работе по модели FBO в значительной степени ориентированы на данные о складских остатках, оборачиваемости и отгрузках. Эти показатели полезны для оперативного управления запасами, однако сами по себе не всегда позволяют корректно определить территориальную структуру фактического спроса. Отсутствие в стандартной отчётности явных сведений о конечном регионе доставки затрудняет принятие обоснованных решений о распределении товара по складам. Это формирует потребность в аналитическом инструменте, который позволял бы использовать доступные данные для более точной интерпретации спроса и поддержки решений по планированию поставок.

1.5. Применение методов искусственного интеллекта в электронной коммерции

В последние годы методы искусственного интеллекта получили широкое распространение в электронной коммерции, в том числе в деятельности продавцов на маркетплейсах. Однако на практике их использование чаще всего связано не с управленческой аналитикой, а с решением прикладных операционных задач.

Параллельно в научной литературе формируется концепция многоагентных систем поддержки принятия решений для электронной коммерции, объединяющих эконометрические модели и алгоритмы машинного обучения с языковыми моделями для генерации аналитических рекомендаций [14]. Такой подход рассматривается как одно из перспективных направлений развития инструментов поддержки управленческих решений на маркетплейсах.

К числу наиболее распространённых направлений применения искусственного интеллекта относятся генерация описаний товаров, подготовка SEO-текстов для карточек товара, создание рекламных формулировок, обработка отзывов покупателей и формирование ответов на вопросы клиентов. Подобные функции уже реализованы во многих сервисах, предназначенных для продавцов маркетплейсов, и активно используются для ускорения рутинных операций и повышения качества коммуникации с покупателями.

Наличие на рынке сервисов, ориентированных на автоматическое создание текстового контента для маркетплейсов, подтверждает, что языковые модели стали заметным элементом инфраструктуры электронной коммерции. Вместе с тем основная область их практического применения по-прежнему связана преимущественно с генерацией и обработкой текстов.

Использование больших языковых моделей для интерпретации аналитических результатов в инструментах для продавцов маркетплейсов пока распространено значительно меньше [8, 11]. В большинстве существующих решений аналитическая информация представляется в виде таблиц, графиков и набора количественных показателей, интерпретация

которых остаётся на стороне пользователя. Это требует времени и определённой аналитической подготовки, особенно при широком ассортименте, работе на нескольких площадках и необходимости регулярно анализировать большое количество товарных позиций.

Интеграция большой языковой модели в состав аналитической системы позволяет перейти от простого отображения числовых показателей к их содержательной интерпретации. На основе рассчитанных метрик модель может выявлять проблемные товарные позиции, указывать на рост логистических или рекламных расходов, фиксировать снижение маржинальности и формулировать рекомендации в понятной для пользователя текстовой форме. Такой подход не заменяет расчётную часть системы, а дополняет её, повышая удобство практического использования результатов анализа.

Преимуществом подобного решения является снижение порога восприятия аналитической информации. Пользователь получает не только значения показателей, но и их интерпретацию, представленную в прикладной и доступной форме. В то же время качество рекомендаций зависит от качества исходных данных, структуры запроса к модели и корректности представления количественных показателей.

В рамках данной работы искусственный интеллект рассматривается не как инструмент генерации карточек товаров или автоматизации коммуникации с клиентами, а как вспомогательный модуль интерпретации аналитических результатов. На основе рассчитанных показателей маржинальности, рекламных расходов и динамики продаж он формирует краткие сигналы на главной странице дашборда, а также более развёрнутые комментарии в разрезе отдельных маркетплейсов.

Выводы по главе 1

Проведённый анализ предметной области показал, что деятельность продавца на маркетплейсах характеризуется сложной и неоднородной структурой затрат, что затрудняет оперативную оценку фактической прибыльности товаров. Существенное влияние на итоговый финансовый результат оказывают комиссии маркетплейсов, логистические расходы, скидочные механизмы, рекламные затраты, а также особенности организации складской инфраструктуры. Дополнительную сложность создают различия в подходах маркетплейсов к формированию начислений и отчётности, что затрудняет сопоставление показателей при одновременной работе на нескольких площадках.

Анализ подходов к расчёту юнит-экономики показал, что они различаются по степени оперативности и точности. Использование прогнозных данных позволяет быстрее получать оценочные показатели, однако расчёт на основе фактических начислений обеспечивает более достоверное представление о маржинальности и структуре расходов по каждому SKU. В связи с

этим в рамках данной работы в качестве методологической основы выбран подход, основанный на фактических данных маркетплейсов.

Также установлено, что при работе по модели FBO существенное значение имеет задача планирования поставок и распределения товарных запасов по складской сети маркетплейса. Эффективность таких решений влияет на скорость доставки, уровень локализации товара, логистические издержки и показатели продаж. Вместе с тем существующие подходы к планированию поставок часто опираются на данные об оборачиваемости и продажах по складам отгрузки, которые не позволяют в полной мере определить географию фактического спроса.

Проведённый обзор существующих аналитических решений показал, что представленные на рынке инструменты, как правило, ориентированы на решение отдельных задач, таких как внешняя аналитика, агрегация отчётности, расчёт прибыли или интеграция с учётными системами. При этом решения, сочетающие мультиплатформенный расчёт юнит-экономики на основе фактических начислений, анализ географии спроса для поддержки планирования поставок и интерпретацию результатов с использованием языковой модели, распространены ограниченно.

Таким образом, разработка специализированного аналитического приложения, объединяющего указанные функции в рамках единой системы, является обоснованной и актуальной. Во второй главе рассматриваются требования к разрабатываемой системе и основные принципы её проектирования.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ И АРХИТЕКТУРА СИСТЕМЫ

2.1. Требования к системе

На основе анализа предметной области и выявленных ограничений существующих решений были сформулированы функциональные и нефункциональные требования к разрабатываемой системе.

Функциональные требования определяют основные возможности, которые система должна предоставлять пользователю. Система должна автоматически получать данные о продажах, начислениях и рекламных расходах через API маркетплейсов Ozon и Wildberries без необходимости ручной выгрузки отчётов из личных кабинетов. Для каждого SKU должен рассчитываться набор ключевых показателей юнит-экономики, включая нетто-выручку, себестоимость, маржу, маржинальность и долю рекламных расходов. Расчёт должен быть доступен как минимум за два периода: предыдущий день и период с начала текущего месяца.

Система должна формировать сводную аналитику как по каждой площадке отдельно, так и по всем площадкам в совокупности, обеспечивая возможность сопоставления показателей. На уровне отдельного SKU пользователь должен иметь доступ к детализации структуры расходов, включая основные статьи удержаний со стороны маркетплейса.

Отдельная группа функциональных требований связана с поддержкой планирования поставок. Система должна анализировать продажи не только в разрезе складов отгрузки, но и с учётом регионов доставки. На этой основе необходимо выявлять устойчивые магистральные потоки, то есть ситуации, при которых спрос в одном регионе систематически обслуживается со складов, расположенных в других регионах. Кроме того, ассортимент должен классифицироваться по методу ABC на основе объёма продаж. Для каждого склада система должна формировать рекомендованный объём поставки с учётом целевого запаса в днях продаж и кратности упаковки товара.

Дополнительным функциональным требованием является наличие AI-компонента, формирующего текстовые рекомендации на основе рассчитанных метрик. Предполагается двухуровневый формат представления рекомендаций: краткие сигналы на главном экране для быстрой оценки ситуации и более развёрнутый анализ на страницах отдельных площадок.

Нефункциональные требования определяют качественные характеристики системы. Время загрузки данных при первичном обращении должно оставаться приемлемым с учётом ограничений на частоту запросов к API маркетплейсов. При повторных обращениях должны использоваться механизмы кэширования. Интерфейс системы должен быть понятным пользователю и не требовать специальной технической подготовки. Архитектура должна быть

модульной и обеспечивать возможность дальнейшего расширения, включая подключение новых площадок без существенной переработки существующих компонентов.

2.2. Архитектура решения

Разрабатываемая система построена по модульному принципу, при котором отдельные функциональные блоки реализованы в виде самостоятельных программных компонентов. Такой подход обеспечивает слабую связанность между частями системы, упрощает тестирование и облегчает последующее расширение решения.

В архитектуре системы можно выделить четыре основных уровня: уровень интеграции с данными, уровень бизнес-логики, уровень кэширования и уровень представления.

Уровень интеграции с данными отвечает за взаимодействие с внешними API маркетплейсов. Для каждой площадки реализован отдельный компонент, инкапсулирующий особенности конкретного API, включая формирование запросов, обработку пагинации, соблюдение лимитов по частоте обращений, обработку ошибок и выполнение повторных запросов при временных сбоях. Для Ozon используются Seller API для получения транзакционных данных и информации об остатках, а также Performance API для извлечения данных о рекламных расходах. Для Wildberries используются Sales API для получения ежедневных данных о продажах, API продвижения для рекламной статистики, а также методы, связанные с получением данных об остатках и складской инфраструктуре.

Уровень бизнес-логики выполняет обработку исходных данных и их преобразование в аналитические показатели. Компоненты данного уровня реализуют расчёт юнит-экономики по каждому SKU, агрегирование показателей по площадкам и периодам, а также процедуры, связанные с анализом поставок. Для расчёта используются данные о начислениях, продажах, рекламных расходах и себестоимости, загружаемой из прайс-листа. Подсистема планирования поставок анализирует соотношение между регионами спроса, остатками на складах и текущей конфигурацией складской сети, после чего формирует рекомендации по объёмам отгрузки.

Отдельный компонент бизнес-логики связан с AI-аналитикой. Его задача состоит в формировании структурированного контекста на основе уже рассчитанных метрик и передаче этого контекста языковой модели для генерации текстовых рекомендаций.

Уровень кэширования предназначен для временного хранения загруженных и рассчитанных данных. Наличие данного уровня позволяет сократить число повторных обращений к API маркетплейсов и ускорить навигацию пользователя между разделами приложения. При необходимости пользователь может инициировать принудительное обновление данных через интерфейс системы.

Уровень представления отвечает за визуализацию результатов и взаимодействие с пользователем. Интерфейс системы реализован в виде веб-приложения с несколькими

страницами, каждая из которых соответствует отдельной функциональной области: общей сводке, аналитике по Ozon, аналитике по Wildberries и блоку планирования поставок.

Поток данных в системе организован следующим образом. При обращении пользователя к определённому разделу интерфейс запрашивает данные из кэша. Если необходимые данные отсутствуют либо срок их актуальности истёк, инициируется обращение к бизнес-логике, которая, в свою очередь, получает исходные данные через интеграционный уровень. После выполнения расчётов результаты сохраняются в кэше и отображаются пользователю. Конфигурационные параметры системы, включая API-ключи, идентификаторы аккаунтов, пути к файлам и расчётные параметры, вынесены в отдельный конфигурационный модуль.

2.3. Модель данных

Одной из ключевых задач при проектировании системы является приведение данных разных маркетплейсов к единой структуре. Ozon и Wildberries используют различные внутренние идентификаторы товаров, различную терминологию и неодинаковую степень детализации отчётности. Поэтому для построения единой аналитической модели необходимо определить общий признак связи.

В качестве такого признака выбран артикул поставщика, соответствующий внутреннему коду товара в учётной системе продавца. На стороне Ozon данный идентификатор передаётся в виде `offer_id`, на стороне Wildberries, в виде `supplierArticle`. Хотя один и тот же товар на площадках имеет различные внутренние идентификаторы, именно артикул поставщика позволяет связать данные разных систем между собой и использовать единый ключ при расчётах.

Себестоимость товара хранится в отдельном прайс-листе, содержащем соответствие между артикулом поставщика и закупочной ценой. Этот файл загружается при инициализации системы и используется при расчёте юнит-экономики. Такой подход позволяет обновлять значения себестоимости без внесения изменений в программный код.

После загрузки данные о начислениях, продажах и расходах приводятся к унифицированной структуре, включающей артикул поставщика, наименование товара, количество реализованных единиц, сумму к перечислению продавцу, детализацию удержаний по основным статьям и рекламные расходы. Для подсистемы поставок дополнительно используются данные об остатках на складах и сведения о продажах с указанием региона доставки.

Рекламные данные поступают из отдельных API. Для Ozon Performance API предоставляет статистику по рекламным кампаниям с детализацией до SKU. Для Wildberries API продвижения предоставляет ежедневные показатели по кампаниям. Поскольку в Wildberries одна рекламная кампания может охватывать несколько товаров, задача корректного распределения рекламных затрат по SKU требует отдельной логики сопоставления. В текущей реализации расходы

учитываются только по тем кампаниям, для которых возможно однозначное отнесение затрат к конкретной товарной позиции.

2.4. Проектирование AI-компонента

AI-компонент спроектирован как вспомогательная надстройка над расчётной подсистемой. Его задача состоит не в выполнении самостоятельных вычислений, а в интерпретации уже рассчитанных показателей и представлении выводов в текстовой форме.

Архитектура AI-компонента имеет двухуровневую структуру. Первый уровень предназначен для формирования кратких сигналов, отображаемых на главной странице системы. Такие сигналы позволяют пользователю быстро зафиксировать наиболее важные наблюдения, например наличие SKU с отрицательной маржинальностью, рост доли рекламных расходов или ухудшение показателей по сравнению с предыдущим периодом. Второй уровень предназначен для более развёрнутой аналитики на страницах отдельных площадок. Здесь формируется текст, содержащий общую оценку ситуации, выделение проблемных и наиболее эффективных позиций, а также возможные рекомендации по дальнейшим действиям.

Формирование запроса к языковой модели осуществляется программно. Компонент AI-аналитики собирает контекст из рассчитанных данных, включая агрегированные показатели за период, перечень SKU с наихудшей маржинальностью, перечень позиций с высокой рекламной нагрузкой и сравнение текущих значений с предыдущим периодом. Далее этот контекст преобразуется в структурированное текстовое представление и передаётся в языковую модель вместе с системными инструкциями, определяющими роль модели и формат ответа.

Выбор конкретной языковой модели определяется доступностью API, стоимостью использования и требованиями к интеграции. В текущей реализации используется GigaChat. При этом архитектура компонента спроектирована таким образом, чтобы замена модели затрагивала только слой взаимодействия с API и обработки ответа, не требуя изменения остальной логики системы.

При проектировании AI-компонента необходимо учитывать ограничения языковых моделей при работе с количественными данными. Возможны ошибки интерпретации абсолютных значений, неточности в выводах и избыточно общие рекомендации. Для снижения вероятности подобных ошибок в структуре промпта фиксируются формат данных, единицы измерения, валюта и желаемый формат ответа.

2.5. Выбор технологического стека

Выбор технологического стека определялся необходимостью обеспечить относительно быструю разработку прототипа, удобство работы с API маркетплейсов, развитые средства обработки данных и достаточные возможности для визуализации аналитической информации.

В качестве основного языка программирования выбран Python. Данный выбор обусловлен широкой экосистемой библиотек для обработки данных, выполнения HTTP-запросов, интеграции с внешними API и построения аналитических приложений. Кроме того, Python широко используется в задачах автоматизации и аналитики, что упрощает поддержку и дальнейшее развитие решения.

Для обработки табличных данных используется библиотека *pandas* [7, 16]. Она предоставляет необходимые средства фильтрации, агрегации, объединения и группировки данных, что делает её удобной основой для реализации расчётной логики.

Взаимодействие с API маркетплейсов реализовано с использованием библиотеки *requests*. Данный инструмент позволяет гибко управлять запросами, заголовками, параметрами авторизации и обработкой ответов, что особенно важно при работе с различающимися API Ozon и Wildberries.

Для построения пользовательского интерфейса выбран фреймворк *Streamlit*. Его использование позволяет создавать интерактивные веб-приложения непосредственно на Python без разработки отдельного фронтенда. Это ускоряет создание аналитического дашборда и даёт возможность сосредоточиться на логике обработки и визуализации данных. Несмотря на определённые ограничения в глубокой кастомизации интерфейса, возможностей *Streamlit* достаточно для реализации прикладной аналитической системы [15].

Для визуализации данных применяется библиотека *Plotly*, обеспечивающая построение интерактивных графиков и диаграмм. Её интеграция со *Streamlit* позволяет удобно отображать структуру расходов, сравнение товарных позиций и динамику ключевых показателей.

Конфигурационные параметры системы, включая API-ключи, идентификаторы аккаунтов, пути к файлам и расчётные настройки, вынесены в отдельный конфигурационный модуль. Это упрощает адаптацию системы к изменению среды эксплуатации и снижает связанность между конфигурацией и программной логикой.

Выводы по главе 2

В данной главе были сформулированы требования к разрабатываемой аналитической системе и обоснованы основные проектные решения. Определена модульная архитектура, включающая уровни интеграции с внешними данными, бизнес-логики, кэширования и представления. Спроектирована единая модель данных, обеспечивающая объединение информации с разных маркетплейсов на основе артикула поставщика. Описан принцип построения AI-компонента как подсистемы интерпретации результатов аналитики. Кроме того, обоснован выбор технологического стека, соответствующего задачам разработки мультиплатформенного аналитического приложения.

ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ АНАЛИТИЧЕСКОГО ПРИЛОЖЕНИЯ ALCHEMY

Приложение Alchemy реализовано как серверное аналитическое веб-приложение на Python с интерфейсом на базе фреймворка *Streamlit*, модулем локального хранения, прослойкой интеграции с API маркетплейсов, расчётными модулями юнит-экономики и дополнительным AI-слоем интерпретации результатов. Архитектурно система включает не только расчёт маржинальности, но и управление пользователями, каталогом, API-ключами, кэширование данных, рекламную аналитику и сценарии планирования поставок. В настоящей главе описывается программная реализация системы по уровням: общая структура проекта, получение данных через API, расчёт юнит-экономики, AI-компонент, пользовательский интерфейс и типовой сценарий работы пользователя.

3.1. Общая структура проекта

Приложение собрано по модульному принципу. Точкой входа выступает файл *app.py*: он инициализирует базу данных, проверяет авторизацию, поднимает боковую навигацию, читает агрегированные данные из кэша и отрисовывает страницы аналитики. Отдельно выделены модули хранения (*db.py*, *crypto.py*, *user_context.py*), загрузки и кэширования данных (*data_loader.py*), интеграции каталогов (*catalog_fetcher.py*), расчёта маржи по площадкам (*ozon_margin.py*, *wb_margin.py*, *ym_margin.py*), а также служебные страницы интерфейса (*pages/settings_api.py*, *pages/settings_price.py*, *pages/admin_users.py*). В коде также присутствуют отдельные файлы для поставок, цен и промо-аналитики, которые подгружаются через *data_loader.py* и *app.py*.

Исходный код разработанного приложения, расположен в репозитории GitHub: <https://github.com/KazantsevAndey/alchemy>

Структура исполняемых модулей репозитория показана на Рисунке 3.1. Состав восстановлен по точке входа, импортам и вызовам `data_loader.refresh_all_data(...)`.

```

alchemy/
├─ app.py
├─ data_loader.py
├─ auth.py
├─ db.py
├─ crypto.py
├─ user_context.py
├─ catalog_fetcher.py
├─ ozon_margin.py
├─ wb_margin.py
├─ ym_margin.py
├─ ozon_supply.py
├─ wb_supply.py
├─ ym_supply.py
├─ ozon_prices.py
├─ ozon_promos.py
├─ wb_promos.py
├─ utils/
│   └─ price_loader.py
├─ pages/
│   └─ settings_api.py
│   └─ settings_price.py

```

Рисунок 3.1 – Структура исполняемых модулей репозитория Alchemy

Назначение основных модулей и их публичные интерфейсы сведены в Таблице 3.1. Содержание Таблицы сформировано по фактическим вызовам и сигнатурам модулей репозитория.

Таблица 3.1 – Состав модулей приложения Alchemy

Модуль	Назначение	Основные файлы	Интерфейсы	Примечания
Ядро интерфейса	Отрисовка страниц, фильтров, графиков, карточек и AI-блоков	<i>app.py</i>	<i>streamlit run app.py</i>	Главная точка входа
Кэш и оркестрация ETL	Загрузка данных из API, запись в pickle-кэш, обновление метаданных	<i>data_loader.py</i>	<i>refresh_all_data</i> <i>load</i> <i>save</i>	Пер-пользовательский кэш
Авторизация и доступ	Логин, сессия, таймаут, роли	<i>auth.py</i> <i>pages/admin_users.py</i>	<i>login_page</i> <i>is_authenticated</i> <i>render</i>	Есть админский режим
Хранилище и секреты	Пользователи, ключи, каталог, цены, шифрование секретов	<i>db.py</i> <i>crypto.py</i> <i>user_context.py</i>	<i>CRUD-функции</i> + <i>Fernet</i>	Ключи шифруются локально
Каталоги и справочники	Получение товарных карточек из API и объединение по артикулу	<i>catalog_fetcher.py</i> <i>pages/settings_price.py</i>	<i>fetch_*_catalog</i> <i>merge_catalogs</i>	Единый ключ, артикул
Расчётная подсистема	Маржинальность, ДРР, сводки и unit economics	<i>ozon_margin.py</i> <i>wb_margin.py</i> <i>ym_margin.py</i>	<i>build_final</i> <i>calc_summary</i> <i>calc_unit_economics</i>	Разная логика по площадкам

Общая архитектура системы показана на Рисунке 3.2. Пользователь работает только с app.py, тогда как загрузка данных, расчёты, база данных и AI-компонент вынесены в отдельные слои.

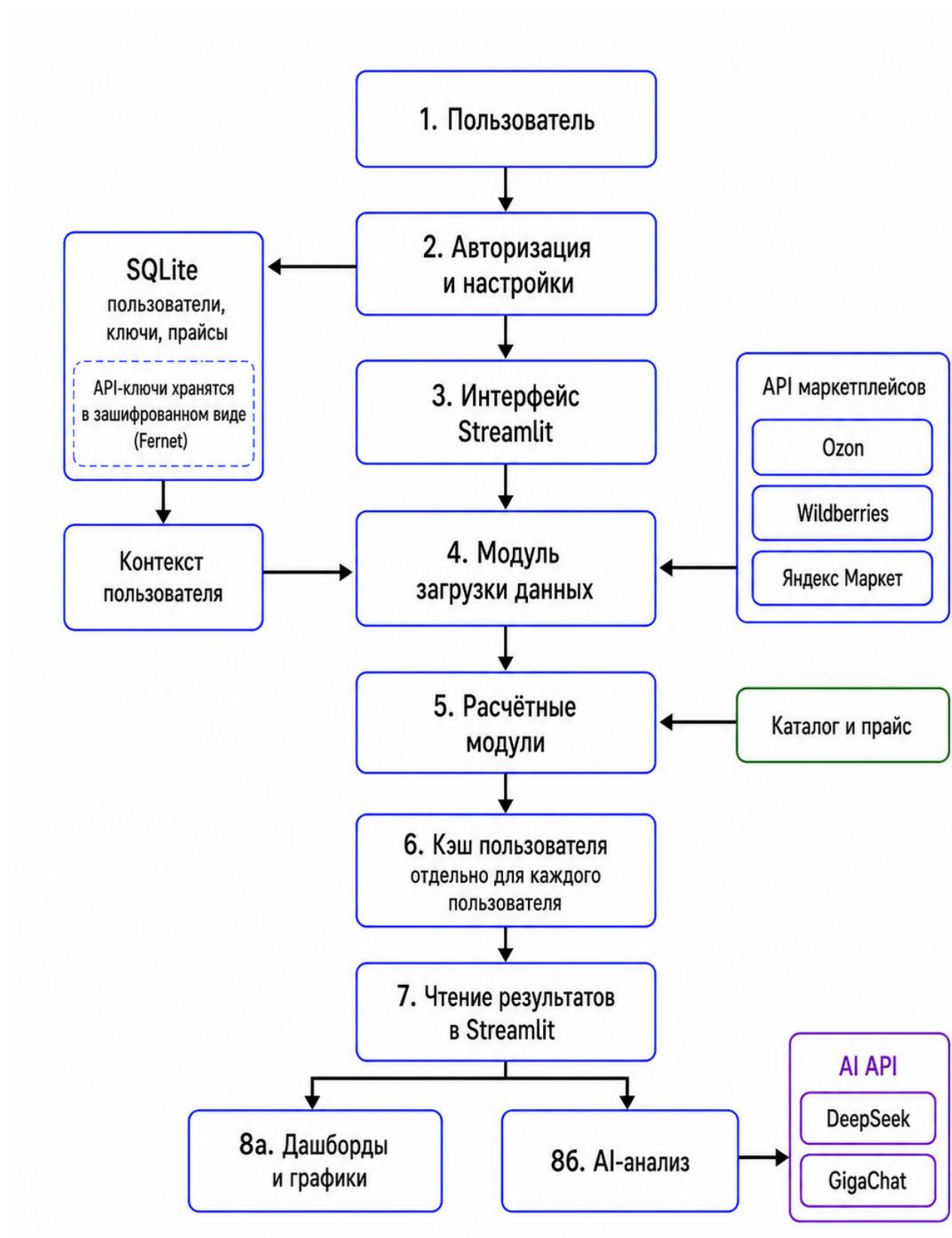


Рисунок 3.2 – Общая архитектура системы Alchemy

Технологический стек в текущей сборке включает *streamlit*, *pandas*, *plotly*, *requests*, *openpyxl*, *numpy*, *bcrypt*, *cryptography*. При этом *app.py* использует также *openai* и SDK *GigaChat*: эти библиотеки задействованы в коде, но не перечислены в *requirements.txt*. Указанное расхождение зафиксировано как ограничение текущей версии и подлежит устранению на этапе подготовки к воспроизводимому развёртыванию.

3.2. Реализация получения данных через API маркетплейсов

Получение данных построено вокруг модуля *data_loader.py*, который выступает оркестратором загрузки. Он определяет даты «вчера» и «с начала месяца», загружает прайс или каталог, последовательно вызывает специализированные функции по площадкам, складывает результаты в пользовательский кэш *data/user_{id}/cache/* и фиксирует метаданные обновления. На выходе формируются именованные объекты кэша: *oz_final_m*, *wb_agg_y*, *ym_margin_m*, *stock_turnover*, *oz_prices*, *wb_promos_dash* и другие. Этот набор ключей и есть фактическая модель обмена данными между серверной частью и интерфейсом.

3.2.1. Интеграция с Ozon

Основной финансовый поток для Ozon реализован в модуле *ozon_margin.py*. Модуль использует Seller API POST */v3/finance/transaction/list* для транзакций и Performance API для рекламы: получение токена POST */api/client/token*, список кампаний GET */api/client/campaign*, постановку асинхронного отчёта POST */api/client/statistics*, ожидание готовности GET */api/client/statistics/{uuid}* и скачивание архива отчёта GET */api/client/statistics/report?UUID=...* Для каталога используются POST */v2/product/list* и POST */v3/product/info/list*, а страница API-ключей дополнительно тестирует Seller API вызовом POST */v2/category/tree*. Авторизация в Seller API строится по заголовкам Client-Id и Api-Key, а в Performance API, через bearer-токен. В модуле реализованы пагинация, пакетная отправка кампаний по 10 идентификаторов, ожидание готовности отчёта, а также повторные запросы при ответе HTTP 429 [1, 2].

3.2.2. Интеграция с Wildberries

В модуле *wb_margin.py* применяются как минимум три API-контура. Финансовые данные загружаются из Statistics API GET */api/v5/supplier/reportDetailByPeriod*; рекламные расходы, по цепочке Promotion API GET */adv/v1/promotion/count* и GET */adv/v3/fullstats*; каталог, через Content API POST */content/v2/get/cards/list*. Авторизация унифицирована через заголовок Authorization. На уровне обработки ошибок реализован явный сценарий для HTTP 429: в статистическом отчёте, ожидание до 65 секунд между страницами и повторный запрос; в рекламном блоке, ожидание 20 секунд и повтор для пачки кампаний. Логика загрузки рассчитана на большие выгрузки: параметр *limit=100000* для отчёта и пакетная обработка рекламы [3].

3.2.3. Интеграция с Яндекс Маркетом

В коде Яндекс Маркета достоверно видны два уровня интеграции. Каталог подгружается через `POST /businesses/{business_id}/offer-mappings`, а тест подключения в настройках выполняется запросом `GET /campaigns/{campaign_id}`. В `data_loader.py` дополнительно вызываются `ym_margin.load_services_report` и `ym_supply.compute_ym_supply_data`, что указывает на отдельный сервисный слой для отчётов и поставок. Детали низкоуровневой реализации модулей Яндекс Маркета в данной версии работы реконструированы по вызовам и интерфейсам [4].

3.2.4. Хранение секретов

Пользователь вводит ключи на странице `settings_api.py`; они шифруются вызовом `crypto.encrypt(...)`, сохраняются в локальном хранилище и затем извлекаются через `user_context.get_user_credentials(...)`. Шифрование выполнено алгоритмом Fernet; ключ берётся либо из переменной окружения `ALCHEMY_FERNET_KEY`, либо создаётся и сохраняется в файл `data/.fernet.key`. Это означает, что секреты защищены «в покое», но безопасность решения зависит и от защищённости файловой системы хоста, на котором лежит сам Fernet-ключ. Указанный компромисс характерен для прототипа и подлежит фиксации как граница экспериментальной версии.

Типичные входные данные для API-модулей, даты (`dateFrom`, `dateTo`), идентификаторы кампаний, токены и прайс-лист с колонками Артикул, Ozon SKU ID, Наименование, Цена в рублях. Типичные выходные данные, объекты `DataFrame` с транзакциями, сводками, разрезом SKU, рекламными расходами и агрегатами по поставкам. Ограничения этой части системы следующие: загрузка выполнена последовательно, а не асинхронно; для некоторых API-веток используются жёстко заданные паузы `sleep`; при сетевых ошибках код чаще печатает ошибку в консоль, чем поднимает формализованное исключение на уровень интерфейса; часть модулей подгружается по факту наличия ключей, поэтому поведение системы может различаться у пользователей с разной конфигурацией аккаунтов.

3.3. Реализация логики расчёта юнит-экономики

Расчёт юнит-экономики в приложении реализован с учётом различий в структуре отчётности маркетплейсов. При этом на уровне методики используется единая экономико-математическая логика: для каждой товарной позиции определяется фактическая база начислений, затем рассчитываются нетто-выручка, себестоимость, прибыль, маржинальность и доля рекламных расходов. Площадочные особенности связаны не с изменением экономического смысла показателей, а с разным составом исходных данных, доступных через API и отчёты Ozon, Wildberries и Яндекс Маркета.

3.3.1. Общая расчётная модель

Для формального описания расчётной части используются унифицированные обозначения. Основные показатели и зависимости, применяемые при расчёте юнит-экономики по SKU, представлены в Таблице 3.2.

Таблица 3.2 – Унифицированная логика расчёта юнит-экономики по SKU

Показатель	Обозначение	Расчётная зависимость	Экономический смысл
Фактическая база начислений	R_{base}	Определяется по данным отчёта маркетплейса	Денежная база, от которой строится расчёт по товарной позиции
Нетто-выручка по SKU	R_{net}	$R_{net} = R_{base} - C_{direct}$	Выручка после вычета расходов, которые могут быть прямо отнесены к SKU
Прямые расходы по SKU	C_{direct}	$C_{direct} = C_{log} + C_{loyalty} + C_{other}$	Логистика, программа лояльности и иные расходы, относимые к товару
Себестоимость реализованных товаров	C_{goods}	$C_{goods} = Q \times C_{unit}$	Совокупная закупочная себестоимость реализованного объёма товара
Прибыль без учёта рекламы	P	$P = R_{net} - C_{goods}$	Финансовый результат до рекламных расходов
Прибыль с учётом рекламы	P_{adv}	$P_{adv} = R_{net} - C_{goods} - C_{adv}$	Финансовый результат после учёта рекламных расходов
Маржинальность без учёта рекламы	M	$M = P / R_{net} \times 100 \%$	Доля прибыли в нетто-выручке
Маржинальность с учётом рекламы	M_{adv}	$M_{adv} = P_{adv} / R_{net} \times 100 \%$	Доля прибыли после рекламных расходов в нетто-выручке
Доля рекламных расходов	ДРР	$ДРР = C_{adv} / R_{net} \times 100 \%$	Отношение рекламных расходов к нетто-выручке

В Таблице 3.2 показатель прямых расходов отражает только те статьи, которые могут быть корректно связаны с конкретной товарной позицией. Расходы уровня аккаунта, например отдельные удержания, хранение или сервисные списания, не распределяются по SKU автоматически, если в исходных данных отсутствует надёжный признак для такого распределения. Такой подход снижает риск искусственного искажения маржинальности отдельных товаров.

Расшифровка используемых обозначений приведена в Таблице 3.3.

Таблица 3.3 – Расшифровка обозначений расчётной модели

Обозначение	Расшифровка
R_{base}	фактическая база начислений по данным маркетплейса
R_{net}	нетто-выручка по SKU
C_{direct}	прямые расходы, относимые к SKU
C_{log}	логистические расходы
$C_{loyalty}$	расходы на программу лояльности
C_{other}	прочие расходы, прямо относимые к SKU
C_{goods}	себестоимость реализованных товаров

Обозначение	Расшифровка
Q	количество реализованных единиц товара
C _{unit}	себестоимость единицы товара
C _{adv}	рекламные расходы
P	прибыль без учёта рекламы
P _{adv}	прибыль с учётом рекламы
M	маржинальность без учёта рекламы
M _{adv}	маржинальность с учётом рекламы
ДРР	доля рекламных расходов

3.3.2. Особенности применения модели для разных маркетплейсов

Единая расчётная модель адаптируется к структуре отчётности каждой площадки. Это позволяет сохранить сопоставимость ключевых управленческих показателей, не игнорируя различия в доступных источниках данных и детализации начислений.

Таблица 3.4 – Адаптация расчётной модели к данным маркетплейсов

Элемент модели	Ozon	Wildberries	Яндекс Маркет
База начислений	Сумма отгрузки по операциям доставки покупателю из транзакционного отчёта Seller API	Сумма продаж за вычетом возвратов по отчёту реализации	Сумма реализации по данным отчётов и API Яндекс Маркета
Прямые расходы	Расходы площадки отражаются в фактических начислениях и детализации транзакций	Логистика, участие в программе лояльности, баллы лояльности и иные расходы, доступные на уровне SKU	Сервисные расходы и удержания площадки, доступные в отчётных данных
Себестоимость	Количество реализованных единиц умножается на себестоимость из прайс-листа	Количество реализованных единиц умножается на себестоимость из прайс-листа	Количество реализованных единиц умножается на себестоимость из прайс-листа
Реклама	Расходы загружаются из Ozon Performance API и сопоставляются с SKU	Расходы загружаются из API продвижения Wildberries и сопоставляются с nm_id или артикулом	Учитываются при наличии данных, которые можно сопоставить с SKU
Ключ сопоставления	Ozon SKU ID и артикул поставщика	nm_id, supplierArticle и внутренний артикул продавца	Идентификатор предложения и внутренний артикул продавца
Ограничение	Итоги уровня аккаунта и SKU могут иметь разную базу агрегации, поэтому требуют отдельной сверки	Хранение и общие удержания не разнятся по SKU без надёжного ключа распределения	Детализация зависит от состава доступных сервисных отчётов и текущей реализации API-модуля

Для Ozon расчёт сосредоточен в функции build_final(...) модуля ozon_margin.py. Из массива транзакций выбираются операции доставки покупателю, после чего данные распределяются до уровня SKU, объединяются с прайс-листом себестоимости и рекламными

расходами. На этом уровне формируются показатели выручки, себестоимости, прибыли, маржинальности, ДРР и маржи с учётом рекламных расходов.

Для Wildberries расчёт выполняется через функции `calc_summary(...)` и `calc_unit_economics(...)` модуля `wb_margin.py`. Первая функция формирует показатели уровня аккаунта, а вторая строит SKU-агрегат, учитывая продажи, возвраты, логистику, расходы на программу лояльности, себестоимость и рекламные расходы. Такое разделение позволяет одновременно видеть общую картину по площадке и детальный разрез по товарным позициям.

Для Яндекс Маркета используется та же управленческая логика расчёта, однако в текущей версии модуль раскрыт более ограниченно по сравнению с Ozon и Wildberries. Основной акцент сделан на подключении каталога, обработке сервисных отчётов и расчёте показателей по доступным данным. Более детальная декомпозиция расходов и рекламных данных рассматривается как направление дальнейшего развития системы.

3.3.3. Сопоставимость итогов между площадками

Несмотря на единую модель расчёта, итоговые показатели между маркетплейсами не являются полностью симметричными. Это связано с тем, что площадки используют разные отчётные сущности, разные правила детализации начислений и разные уровни доступности данных. Поэтому в приложении важно различать два уровня анализа: сверку итогов уровня аккаунта с официальными отчётами маркетплейса и управленческий анализ SKU, в котором часть расходов распределяется только при наличии корректного ключа сопоставления.

В Ozon-сводке итоговые значения уровня аккаунта могут строиться по агрегированной таблице начислений, тогда как SKU-таблица опирается на операции доставки покупателю. В Wildberries часть расходов уровня аккаунта, например хранение и отдельные удержания, не распределяется по SKU. В Яндекс Маркете детализация зависит от состава доступных отчётов и подключённых API-модулей. Поэтому система не сводит все площадки к искусственно одинаковой структуре, а сохраняет площадочные особенности и явно фиксирует ограничения расчёта.

Такой подход позволяет использовать приложение как инструмент управленческой аналитики: пользователь видит сопоставимые показатели прибыли, маржинальности и рекламной нагрузки, но при этом понимает границы точности расчёта для каждой площадки. Это особенно важно при принятии решений о цене, рекламном бюджете, ассортименте и поставках.

3.4. Реализация AI-компонента

AI-компонент в Alchemy выполняет роль слоя интерпретации, а не расчёта. Все управленческие метрики вычисляются детерминированно в расчётных модулях, после чего передаются в языковую модель в виде структурированного контекста. В *app.py* для подготовки

этого контекста реализованы две функции-конструктора: `_build_full_json()` для сводного сигнала и `_build_mp_json(mp)` для анализа отдельной площадки. Первая собирает по дням и месяцу данные Ozon, Wildberries и Яндекс Маркета, дополняет их структурой расходов, списком top-SKU и перечнем низкомаржинальных товаров; вторая сужает этот объект до одной площадки.

В коде явно зафиксированы системные инструкции `_DEEPSEEK_SIGNAL_SYSTEM`, `_SIGNAL_SYSTEM` и `_ANALYSIS_SYSTEM`, в которых задаются роль модели, шкала оценки маржинальности, требования к стилю ответа, ограничения на «воду» и ожидаемый формат JSON для карточки сигнала. Модель не получает сырые таблицы, а работает с заранее нормализованным контекстом и жёстко заданным форматом ответа.

Внешние вызовы реализованы двумя функциями: `_deepseek_call(...)` обращается через OpenAI-совместимый клиент к модели DeepSeek, а `_giga_call(...)`, через SDK GigaChat. На главной странице приложение сначала пытается запросить краткий сигнал у DeepSeek, а при неуспехе использует GigaChat в качестве резервного канала. На страницах конкретных площадок применяется вызов `_giga_call(...)` с более развёрнутым аналитическим промптом. При отсутствии ответа от внешних моделей система не прерывает работу, а переключается на rule-based fallback: `_show_auto_fallback(...)` и `_show_mp_auto_fallback(...)` формируют заключение на основе уже рассчитанных метрик.

Кэширование организовано на двух уровнях. Тяжёлые данные маркетплейсов сначала складываются в pickle-кэш через `data_loader.py`; чтение кэша на уровне интерфейса дополнительно декорировано `@st.cache_data(ttl=300)`. Языковая модель получает уже агрегированные данные и не инициирует повторную загрузку API. Отдельного долговременного кэша LLM-ответов в коде нет; при необходимости его можно добавить по хэшу входного JSON.

Ограничения AI-слоя следующие. Во-первых, ответы модели зависят от ранее рассчитанных метрик и не должны рассматриваться как первоисточник. Во-вторых, текущая реализация использует параметр `verify_ssl_certs=False` в GigaChat-клиенте, что приемлемо для прототипа, но нежелательно для промышленной эксплуатации. В-третьих, в `requirements.txt` не отражены пакет `openai` и SDK GigaChat, хотя они импортируются в приложении; это критично для воспроизводимости.

3.5. Пользовательский интерфейс

После авторизации пользователь попадает в сайдбар с радионавигацией по страницам: «Сводка», Ozon, Wildberries, Яндекс Маркет, «Поставки», «API-ключи», «Прайс-лист» и, для администратора, «Пользователи». На каждой аналитической странице строятся карточки KPI, таблицы, фильтры, интерактивные графики и кнопки запуска AI-анализа. Для визуализации используется библиотека Plotly, а сами страницы оформлены как единый BI-интерфейс с глобальными CSS-стилями.

Главная страница «Сводка» содержит агрегированные карточки по Ozon и Wildberries за вчера и за месяц, кнопку «AI-сигнал», donut-графики структуры расходов и рейтинги top-15 SKU по выручке. Страницы площадок разворачивают этот анализ глубже: показывают карточки по периоду, структуру расходов, юнит-экономику по SKU с фильтрами по маржинальности и количеству, отдельный блок анализа ДРР и кнопку подробного AI-анализа. Интерфейс поддерживает последовательный drill-down от управленческой сводки до уровня конкретного SKU.

Страница API-ключей позволяет вводить, изменять, очищать и тестировать ключи по площадкам и AI-сервисам. Страница себестоимости решает две задачи: формирование каталога из API и массовую загрузку или редактирование закупочных цен. Административная страница поддерживает операции CRUD над пользователями. Таким образом, приложение включает не только аналитические витрины, но и административный контур подготовки данных, что обеспечивает завершённость пользовательского контура от настройки доступа до принятия решения по SKU.

В текущей версии присутствует существенное UX-ограничение: в *app.ru* реализованы дополнительные страницы «Остатки», «Цены», «Акции» и «Цены WB», однако они не включены в основной список *pages* для навигации в сайдбаре. Код этих разделов существует, но в текущем интерфейсе обычный пользователь к ним не попадает. Эти функции квалифицируются как реализованные, но не выведенные в основную навигацию; их включение в маршрутизацию входит в перечень доработок.

Пользовательский поток в приложении раскрывается в двух схемах: ежедневный сценарий работы показан на Рисунке 3.4, а первичная настройка системы - на Рисунке 3.5.

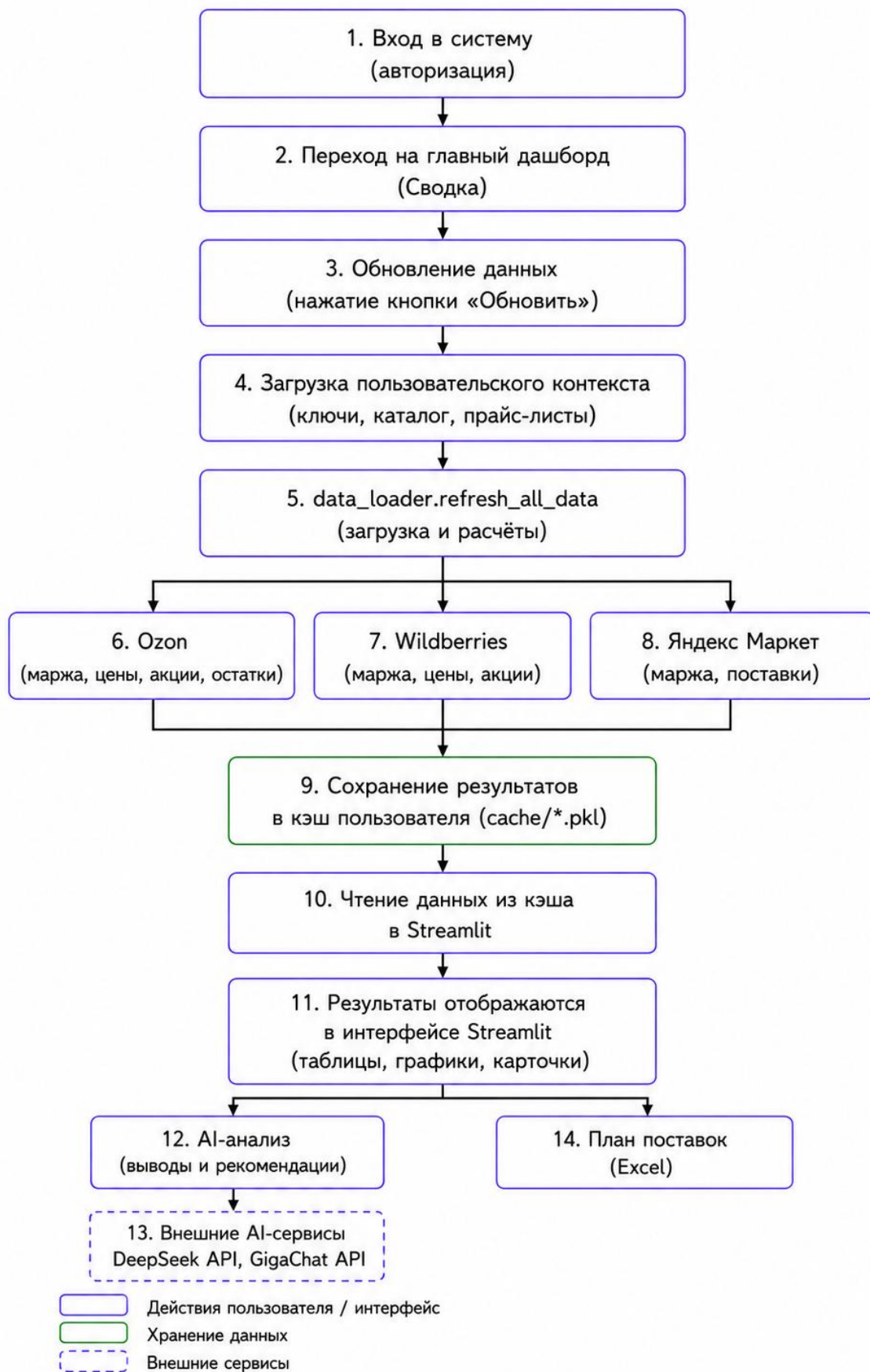


Рисунок 3.4 – Ежедневный сценарий работы с системой

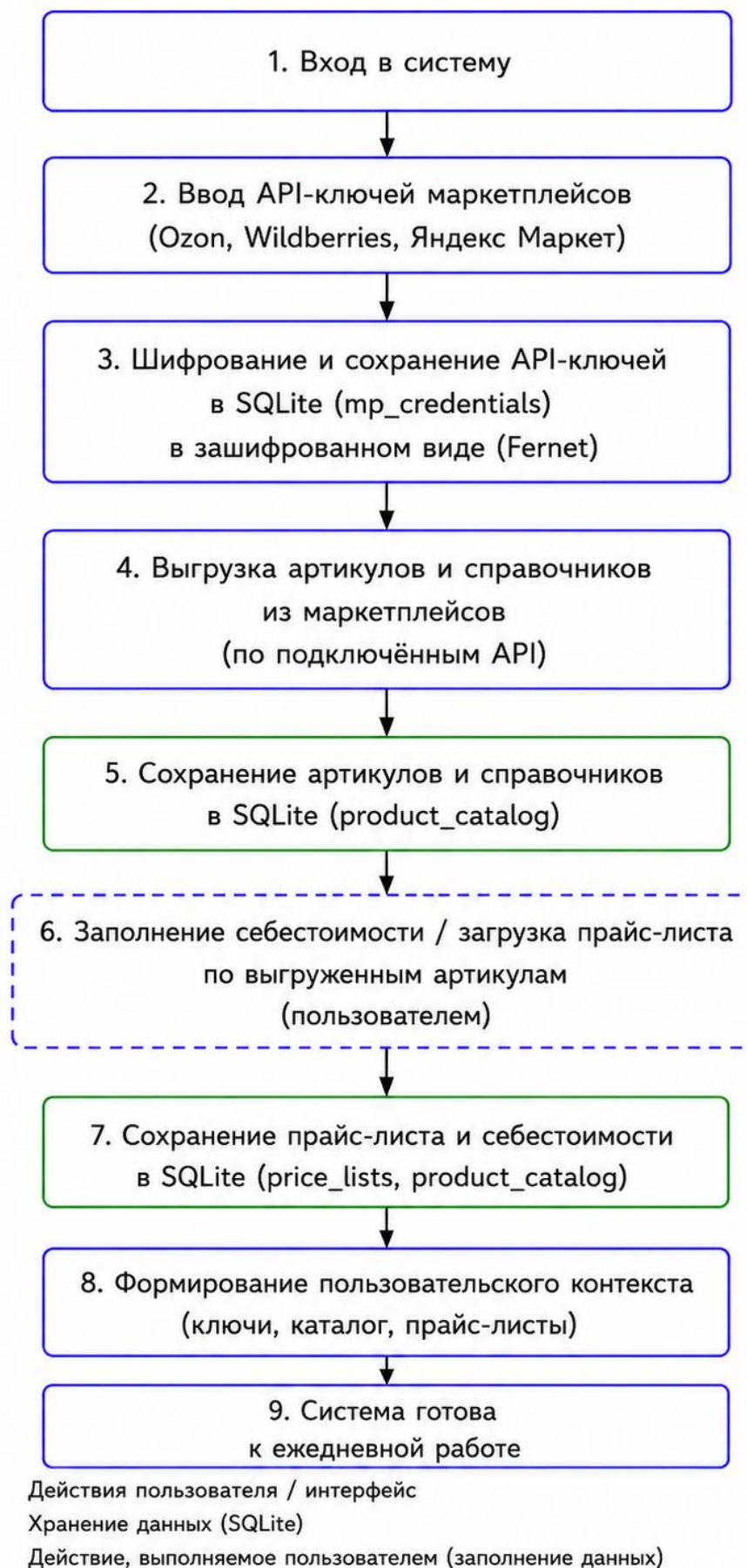


Рисунок 3.5 – Первичная настройка системы

Типичный формат данных, который пользователь видит на уровне интерфейса, это таблицы с колонками по SKU, выручке, марже, ДРР, прибыли, а также таблицы структуры расходов и плана поставок в разрезе кластеров. Большинство таблиц поддерживают фильтрацию; экспорт в Excel встроен непосредственно в сценарии использования, особенно для модулей поставок и Яндекс Маркета.

3.6. Сценарий работы пользователя

Типовой сценарий работы с приложением представляет собой последовательность шагов. Сначала пользователь проходит аутентификацию: проверка сессии идёт через *is_authenticated()*, форма входа рисуется в *login_page()*. После успешного входа в *st.session_state* сохраняются идентификатор пользователя, имя, компания и роль; сессия имеет таймаут восемь часов. Это обеспечивает работу приложения в многопользовательском режиме, а не в режиме локального оператора.

Далее пользователь настраивает API-ключи и при необходимости каталог и себестоимость. Ключи вводятся на специальной странице, а каталог можно либо построить автоматически из API нескольких площадок, либо загрузить закупочные цены из файла Excel. После этого запускается команда «Загрузить данные» или «Обновить данные»: приложение вызывает *refresh_all_data(...)*, получает данные из маркетплейсов, сохраняет их в кэш и перезагружает интерфейс. Этот момент является границей между ETL-частью и блоком потребления аналитики.

После обновления кэша пользователь работает в режиме аналитического потребления. Он открывает сводку, оценивает состояние бизнеса за вчера и за месяц, вызывает AI-сигнал, переходит на страницу конкретной площадки, фильтрует SKU по марже или ДРР, выявляет проблемные позиции, после чего переходит в блок поставок и формирует Excel-план дозагрузки по кластерам. Указанный маршрут полностью соответствует целям выпускной квалификационной работы: переход от исходных фактических данных к управленческому решению по маржинальности, рекламе и запасам.

Типовой поведенческий кейс выглядит следующим образом. Менеджер открывает сводку, обнаруживает низкую маржу по одной из площадок, переходит в детальную страницу этой площадки, фильтрует товары с маржой ниже 15 %, изучает таблицу ДРР и AI-комментарий, после чего корректирует цену либо рекламные настройки или переходит в модуль поставок и формирует Excel-план перемещений. Этот кейс демонстрирует не только наличие отдельных экранов, но и их логическую связность в едином пользовательском потоке.

3.7. Проверка корректности расчётов и апробация приложения

Проверка корректности расчётной логики выполнялась на фактических данных компании «Италко Ритейл» за апрель 2026 года. В качестве контрольных источников использовались отчёты маркетплейсов, полученные через электронный документооборот, данные личных кабинетов площадок, API-данные, а также внутренние управленческие отчёты компании и прайс-лист себестоимости товаров.

Основная проверка проводилась для финансовых показателей, связанных с продажами, возвратами, начислениями, удержаниями, рекламными расходами и себестоимостью товаров. Результаты, рассчитанные приложением, сопоставлялись с итоговыми значениями в отчётах маркетплейсов и данными, применяемыми сотрудниками компании при управленческом анализе. Такой подход позволил проверить не только техническую корректность загрузки данных, но и соответствие расчётной логики реальным бизнес-процессам продавца.

Проверка выполнялась на нескольких уровнях. На уровне аккаунта сопоставлялись итоговые суммы начислений, продаж, возвратов и расходов по площадкам. На уровне SKU проверялись количество реализованных единиц, сопоставление артикула с прайс-листом себестоимости, расчёт себестоимости реализованного объёма, прибыли и маржинальности. Для рекламных расходов дополнительно проверялась корректность загрузки данных из рекламных кабинетов и их привязка к товарным позициям в тех случаях, когда такая привязка доступна в исходных данных.

Таблица 3.5 – Направления проверки расчётной логики приложения

Направление проверки	Контрольный источник	Что проверялось
Финансовые начисления	Отчёты маркетплейсов через ЭДО, личные кабинеты и API	Продажи, возвраты, удержания, итоговые начисления и расходы площадки
Расчёт по SKU	Исходные отчёты маркетплейсов и прайс-лист себестоимости	Количество реализованных единиц, себестоимость, прибыль и маржинальность
Рекламные расходы	Рекламные кабинеты и API площадок	Сумма рекламных расходов, ДРР и возможность сопоставления расходов с SKU
Пользовательский интерфейс	Рабочие сценарии сотрудников компании	Корректность отображения сводок, таблиц, фильтров, графиков и AI-комментариев
Поставки	Исходные данные по остаткам, продажам и внутренняя логика расчёта приложения	Расчёт потребности, целевого запаса, кратности упаковки и формирование рекомендаций

Результаты проверки показали, что приложение корректно воспроизводит основную логику расчёта управленческой маржинальности по данным маркетплейсов. При этом отдельные расхождения между данными приложения и итоговыми бухгалтерскими документами могут возникать из-за различий в периодизации операций, округлений, особенностей отражения возвратов, а также из-за того, что часть расходов маркетплейса относится к уровню аккаунта и не может быть однозначно распределена по отдельным SKU.

Отдельно следует отметить модуль планирования поставок. В отличие от финансовых показателей, рекомендации по поставкам не имеют прямого контрольного значения в отчётах ЭДО или бухгалтерских документах. Поэтому данный модуль проверялся не путём сопоставления с официальным итоговым отчётом, а через анализ корректности исходной логики: загрузки остатков, учёта продаж, расчёта потребности, применения целевого запаса и кратности упаковки.

В первоначальной версии методологии предполагалось использовать показатель локализации заказов, позволяющий оценивать долю заказов, обслуженных из складского кластера, соответствующего региону спроса. Однако в дальнейшем состав доступных данных маркетплейса был изменён, и прямой расчёт данного показателя стал невозможен. В связи с этим в текущей версии работы модуль поставок рассматривается как инструмент расчётной поддержки планирования, а не как полностью верифицированная модель оптимизации складского распределения.

Приложение развернуто на собственном сервере автора и используется сотрудниками компании «Италко Ритейл» в рабочих аналитических сценариях. Пользователи получают доступ к сводной аналитике по маркетплейсам, анализируют показатели по SKU, выявляют товары с низкой маржинальностью и оценивают влияние рекламных расходов на итоговую прибыльность. Это подтверждает практическую применимость разработанного решения не только как учебного прототипа, но и как инструмента поддержки управленческих решений в действующей компании.

3.8. Ограничения текущей реализации и направления развития

Разработанное приложение представляет собой рабочую серверную версию аналитической системы, применяемую в практических сценариях компании «Италко Ритейл». При этом текущая реализация сохраняет статус развиваемого прикладного решения и имеет ряд ограничений, связанных как с особенностями данных маркетплейсов, так и с этапом зрелости самой системы.

Первое ограничение связано с условиями эксплуатации. Приложение развернуто на собственном сервере автора и используется как рабочий аналитический инструмент, однако пока не является промышленным корпоративным сервисом, полностью встроенным в ИТ-инфраструктуру компании. При дальнейшем развитии системы необходимо формализовать регламенты резервного копирования, мониторинга, управления доступом, обновления версий и сопровождения пользователей.

Второе ограничение обусловлено неоднородностью данных маркетплейсов. Ozon, Wildberries и Яндекс Маркет используют разные форматы отчётности, различную детализацию начислений и разные подходы к представлению расходов. Поэтому расчётные модели по площадкам не могут быть полностью симметричными. В системе это ограничение учитывается

за счёт разделения показателей уровня аккаунта и уровня SKU, а также за счёт отказа от искусственного распределения расходов при отсутствии надёжного ключа привязки к товарной позиции.

Отдельного внимания требует учёт расходов, которые относятся к площадке или аккаунту в целом. К таким расходам могут относиться хранение, отдельные удержания, сервисные начисления и корректировки. Если такие расходы нельзя однозначно отнести к конкретному SKU, они учитываются на агрегированном уровне. Такой подход снижает детализацию SKU-аналитики, но позволяет сохранить корректность расчётной методологии.

Третье ограничение связано с модулем поставок. Его работа зависит от состава данных, предоставляемых маркетплейсами. В первоначальной логике модуля предполагалось использовать показатель локализации заказов, однако после изменения состава доступных данных прямой расчёт этого показателя стал невозможен. В связи с этим текущая версия модуля поставок рассматривается как инструмент расчётной поддержки планирования, основанный на остатках, продажах, целевом запасе и кратности упаковки. Дальнейшее развитие этого направления связано с поиском новых источников данных о региональном спросе и с использованием косвенных признаков для оценки качества распределения запасов.

Четвёртое ограничение связано с различной степенью зрелости интеграций. Наиболее полно в текущей версии реализованы модули Ozon и Wildberries, поскольку именно по этим площадкам доступен наиболее полный набор финансовых и рекламных данных. Интеграция с Яндекс Маркетом включена в архитектуру приложения, однако требует дальнейшего расширения, детализации расчётов и проверки на большем объёме данных.

AI-компонент также имеет прикладные ограничения. Он не выполняет самостоятельных расчётов и не является источником первичных данных. Его функция состоит в интерпретации уже рассчитанных показателей и формировании текстовых рекомендаций для пользователя. Поэтому дальнейшее развитие AI-слоя должно включать контроль качества промптов, логирование ответов модели, настройку правил fallback-аналитики и механизмы объяснения рекомендаций.

Основными направлениями дальнейшего развития приложения являются расширение поддержки Яндекс Маркета, развитие модуля поставок, добавление более строгих тестов расчётных функций, асинхронная загрузка данных из API, улучшение системы кэширования, развитие ролевой модели доступа, настройка резервного копирования и подготовка экспортных управленческих отчётов для руководителя.

Выводы по главе 3

В главе описана программная реализация аналитического приложения Alchemy. Система построена по модульному принципу с разделением на интерфейс, кэш-оркестратор, контур

авторизации и хранения секретов, каталог-сервис и расчётную подсистему по площадкам. Получение данных реализовано через прямую интеграцию с API Ozon, Wildberries и Яндекс Маркета. Расчёт юнит-экономики выполнен отдельно по площадкам с учётом структурных различий первичных данных. AI-компонент реализован как слой интерпретации над уже рассчитанными метриками, с резервированием через rule-based fallback. Дополнительно в главе определены процедуры верификации расчётов, описана апробация на реальных данных и зафиксированы ограничения текущей реализации. Это позволяет рассматривать разработанное приложение как завершённый прототип аналитической системы для поддержки управленческих решений продавца маркетплейсов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была разработана аналитическая система Alchemy, предназначенная для поддержки управленческих решений продавца маркетплейсов. Система решает задачу автоматизированного расчёта маржинальности, анализа структуры затрат, оценки рекламной нагрузки и поддержки планирования поставок на основе данных, получаемых из API маркетплейсов и внутренних справочников компании.

В первой главе была рассмотрена предметная область торговли на маркетплейсах. Анализ показал, что продавец работает в условиях сложной структуры комиссий, логистических расходов, рекламных затрат, скидочных механизмов и возвратов. Было обосновано, что данные о заказах и валовой выручке недостаточны для оценки прибыльности товара, поскольку итоговый финансовый результат формируется на основе фактических начислений площадки.

Во второй главе были сформулированы функциональные и нефункциональные требования к системе, описана модульная архитектура решения, определена единая модель данных и обоснован выбор технологического стека. В качестве ключевого идентификатора для объединения данных разных маркетплейсов был выбран артикул поставщика, что позволяет сопоставлять товарные позиции в Ozon, Wildberries, Яндекс Маркете и внутреннем прайс-листе.

В третьей главе была описана программная реализация приложения. Были рассмотрены модули получения данных через API, расчёта юнит-экономики, кэширования, хранения секретов, пользовательского интерфейса и AI-интерпретации результатов. Отдельное внимание уделено различиям между расчётными моделями Ozon и Wildberries, поскольку эти площадки предоставляют данные в разных форматах и требуют разных алгоритмов обработки.

Практическим результатом работы стала серверная версия веб-приложения, развернутая на собственном сервере автора и используемая сотрудниками компании «Италко Ритейл» в рабочих аналитических сценариях. Приложение позволяет пользователю пройти полный аналитический сценарий: настроить API-ключи, загрузить каталог и себестоимость, получить данные маркетплейсов, рассчитать показатели по SKU, выявить низкомаржинальные позиции, оценить рекламную нагрузку и сформировать управленческие выводы. AI-компонент дополняет расчётную часть, формируя текстовую интерпретацию результатов и снижая трудоемкость анализа для пользователя.

Цель выпускной квалификационной работы достигнута. Разработанное приложение обеспечивает автоматизированную обработку данных маркетплейсов и предоставляет продавцу инструмент для более точной оценки финансового результата по товарным позициям. Факт серверного развёртывания на собственном сервере автора и использования приложения сотрудниками компании подтверждает практическую применимость предложенного решения.

При этом система не заменяет управленческое решение пользователя, а повышает прозрачность данных и сокращает время перехода от отчётности маркетплейсов к прикладным выводам.

Ограничения текущей версии связаны с неодинаковой детализацией данных маркетплейсов, частичной невозможностью распределения отдельных расходов по SKU, последовательной загрузкой API-данных и необходимостью дальнейшей верификации на большем объёме периодов и товарных категорий. Эти ограничения не отменяют работоспособность прототипа, но задают направления дальнейшего развития.

В дальнейшем система может быть расширена за счёт более полной интеграции с Яндекс Маркетом, асинхронной загрузки данных, развития модуля поставок, добавления автоматических тестов расчётной логики, долговременного хранения ответов AI-компонента и формирования управленческих отчётов для руководителя. Таким образом, проект может развиваться от учебного прототипа к прикладному инструменту для малого и среднего бизнеса, работающего на маркетплейсах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ozon Seller API. Официальная документация API для продавцов. URL: <https://docs.ozon.ru/api/seller/> (дата обращения: 25.04.2026).
2. Ozon Performance API. Официальная документация рекламного API. URL: <https://docs.ozon.ru/api/performance/> (дата обращения: 25.04.2026).
3. Wildberries. Официальная документация API для продавцов. URL: <https://dev.wildberries.ru/> (дата обращения: 25.04.2026).
4. Яндекс Маркет. Partner API. Официальная документация. URL: <https://yandex.ru/dev/market/partner-api/> (дата обращения: 25.04.2026).
5. Итоги интернет-торговли в России за 2025 год: аналитический материал Ассоциации компаний интернет-торговли (АКИТ). URL: <https://akit.ru/news/11-5-trln-rublej-akit-podvela-itoqi-internet-torgovli-za-2025-god> (дата обращения: 28.02.2026).
6. Интернет-торговля в России 2026: маркетинговое исследование Data Insight. URL: https://datainsight.ru/DI_eCommerce_2026 (дата обращения: 10.04.2026).
7. Маккинни У. Python и анализ данных / пер. с англ. А. А. Слинкина. 2-е изд. М.: ДМК Пресс, 2020. 540 с. ISBN 978-5-97060-590-5.
8. Феннер М. Машинное обучение с помощью Python для всех. М.: БОМБОРА, 2023. 672 с. ISBN 978-5-04-187899-3.
9. Четыркин Е. М. Финансовая математика: учебник. 10-е изд. М.: Издательский дом «Дело» РАНХиГС, 2011. 392 с. ISBN 978-5-7749-0570-6.
10. Provost F., Fawcett T. Data Science for Business. Sebastopol, CA: O'Reilly Media, 2013.
11. Sharda R., Delen D., Turban E. Analytics, Data Science, and Artificial Intelligence: Systems for Decision Support. Hoboken, NJ: Pearson, 2020.
12. Варнухов А. Ю. Особенности ценообразования на маркетплейсах // Вестник Томского государственного университета. Экономика. 2025. № 69. С. 164–182. DOI: 10.17223/19988648/69/9.
13. Юманов С. А. Оптимизация ценообразования на маркетплейсах // Экономика и парадигма нового времени. 2025. № 5. С. 100–108.
14. Ершов М. А. Экономико-математическое моделирование конкурентоспособности товаров электронной коммерции в архитектуре многоагентной системы поддержки принятия решений // Инновационная экономика: информация, аналитика, прогнозы. 2026. № 3. С. 92–98. DOI: 10.47576/2949-1894.2026.3.3.011.
15. Streamlit Documentation. Streamlit Inc. URL: <https://docs.streamlit.io/> (дата обращения: 25.04.2026).

16. McKinney W. Data Structures for Statistical Computing in Python // Proceedings of the 9th Python in Science Conference (SciPy 2010). Austin, TX, 2010. P. 56–61. DOI: 10.25080/Majora-92bf1922-00a.
17. Аббакумов А. А., Ямашкин С. А., Карнетов В. Г., Рощихин Д. С. Прогнозирование спроса и управление запасами с помощью машинного обучения // Инженерный вестник Дона. 2025. № 6. URL: <https://www.ivdon.ru/ru/magazine/archive/n6y2025/10157> (дата обращения: 25.04.2026).
18. Рыльцева К. В. Формирование предпринимательской стратегии на маркетплейсах // Путеводитель предпринимателя. 2026. Т. 19. № 2. С. 78–86. DOI: 10.24182/2073-9885-2026-19-2-78-86.